step is connected to the next (like the links of a pipe). As an example, let's look at a hypothetical processor that supports two independent stages, Fetch and Execute, as shown in Figure 2.

The first stage fetches an instruction and buffers it. While the second stage is executing the instruction, the first stage takes advantage of any unused memory cycles to fetch and buffer the next instruction. This process speeds up the execution of an instruction.

Now, let's contrast a pipelined processor with a non-pipelined processor.

Non-pipelined processors

Older processors did not use the pipelining feature, so were only able to process one instruction at a time. As shown in Figure 3, we have 4 stages in the pipeline, and one instruction is processed at a time.

As another example, see Figure 4, in which we have five stages in the pipeline. Instruction I1 must completely finish before the processing of Instruction I2 can begin. The total time to execute both the instructions is 10 clock cycles.

Pipelined processor

If a processor supports pipelining, a new instruction can be inserted into the instruction pipeline while other instructions are at other stages of the pipeline. See Figure 5.

As another example, see Figure 6, in which we have five pipeline stages, and five instructions to be executed. As shown in the diagram, Instruction I2 is inserted during Cycle 2, allowing it to go through Stage S1 while Instruction I1 is in Stage S2.

Figure 6 shows five instructions: I1, I2, I3, I4, and I5, all processed using this approach. The total time to process five instructions is only 9 clock cycles. Thus, we can see the increase in instruction throughput for a pipelined processor.

The interface between the stages of a pipeline

A straightforward pipeline consists of a number of stages, one for each sub-task. The stages are decoupled from each other by registers, called latches. As each clock cycle ends, the latches 'gate in' their inputs and forward them to the subsequent stage where the required operation will be performed.

In reality, the latches are extended with multiplexers that select and transfer data from the output of the preceding Execution Units (EUs) to the input of subsequent EUs. Latches between the pipeline stages help preserve the results of each stage so they can be used by the next stage. The various MUXs (multiplexers) provide control functions such as forwarding, exception handling, and the write-back of PC information. The general structure of the pipeline is shown in Figure 7.

Pipeline stages

Now let us understand the various stages in a pipeline. In this article, I have used the RISC pipeline (with five stages)

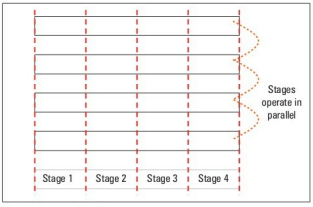

Figure 1: Concept of pipelining

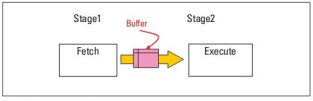

Figure 2: A two-stage pipeline

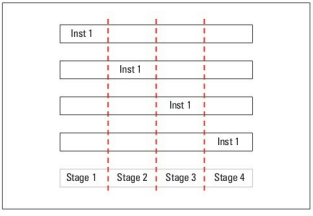

Figure 3: One instruction is processed in a non-pipelined processor

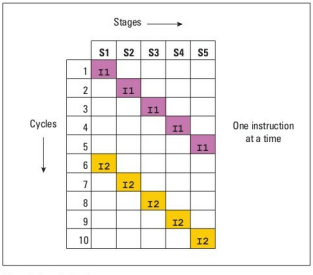

Figure 4: Non-pipelined processor